AISHWARYA AR

Group PM, Vyapar · IIT Madras · aishy.savi@gmail.com · +91-9884299621

How do you use AI in your day-to-day work as a Product Manager?

It's embedded in almost everything I do now. At the most basic level, I use Cursor as my primary IDE, not as a novelty, but as a genuine building environment. I prototype features, write backend logic, build full-stack apps, and generate production-quality code directly. When I need to spec something, I'll often just build the first version instead of writing a document about it. It's faster, and you end up solving real design problems that a PRD never surfaces.

Beyond personal productivity, I've been building **AI-native product experiences** at Vyapar: LLM-based call analytics that improved intent capture by 20% and processed 5x higher volume, generative AI for automated catalog creation that cut merchant onboarding time by 60%, and a business copilot that delivers proactive insights and anomaly detection to merchant partners.

What's your comfort level with the actual technical stack behind AI products: models, agents, RAG, evals?

Very high, and it's not theoretical. I've **personally built and shipped agent systems** that use OpenAI function-calling with 15+ custom tools, RAG pipelines with vector embeddings and Pinecone, multi-agent orchestration with handoffs and shared state machines, MCP server integrations (Google Sheets, Notion, Slack), and eval frameworks using Braintrust with automated dataset generation and multi-dimensional scoring.

I understand the full stack, from how token costs compound across multi-turn conversations, to why you need similarity thresholds and metadata filtering on your RAG retrievals, to why an agent's prompt architecture matters more than the model you pick. When engineers discuss context windows or tool routing, I'm not learning on the fly. I've built these systems myself.

How do you think about building AI agents specifically: what separates a good agent from a chatbot?

Most AI products are chatbots with a system prompt. An agent is fundamentally different: it **acts**. It calls tools, retrieves context, makes decisions about what to do next, and adapts based on what it finds. The design challenge isn't "what should the AI say," it's "what should the AI *do*, in what order, with what information, and when should it stop."

Concretely, that means: designing tool schemas that give the model the right capabilities without overwhelming it. Writing prompts that encode business logic, not just personality. Building state machines that let multiple agents collaborate without losing coherence. And critically, making every decision the agent makes **inspectable**. I've built observability systems that surface token usage, tool call traces, RAG chunks retrieved, and full prompt visibility on every single turn. If you can't see why your agent did what it did, you can't improve it.

Can you walk through how you approach prompt engineering: not the basics, but at a systems level?

Prompt engineering at the systems level is really about **information architecture for an LLM**. You're deciding what the model knows, when it knows it, and how that knowledge is structured.

For example, I built a business advisor agent that needed to serve electronics retailers, restaurants, and apparel businesses, each with completely different mental models. The prompt doesn't just say "be helpful." It encodes: how to mirror the user's language (Hindi, Hinglish, English) by reading their last 2-3 messages. What to do on session start vs. after engagement. That tool results come back in English but the response must match the user's language. That the agent must give one insight per message, never a wall of text. That follow-ups must add new information, never repeat prior advice.

That's not a prompt, it's a **behavioral specification** that happens to be written in natural language. Getting this right is the difference between a demo and a product.

What's your take on evals for AI products? How do you measure quality?

Evals are the most underinvested area in AI product development. Most teams ship on vibes ("it feels better") and then get surprised when the model regresses after a prompt change. I've built eval pipelines using **Braintrust** that automate dataset generation, score responses across multiple dimensions (factual accuracy, tone, tool usage correctness, response length), and run regression tests before any prompt or model change goes live.

The key insight is that AI evals are a **product design problem**, not just an engineering one. You need to decide: what does "good" look like for this agent? Is it accuracy? Is it empathy? Is it knowing when *not* to use a tool? Those are product decisions that determine your scoring rubric. And then you need enough representative examples: edge cases, adversarial inputs, multi-turn conversations, to actually trust the scores. I think the PM who owns the eval framework has more leverage over AI product quality than the one who owns the prompt.

How do you think about MCP and tool-use ecosystems for agents?

MCP (Model Context Protocol) is the most important infrastructure shift happening in AI right now. It standardizes how agents connect to external services. Your agent doesn't need custom integrations for every tool, it discovers capabilities at runtime through a shared protocol. I've integrated MCP servers for **Google Sheets, Notion, and Slack** into agent systems, where the agent connects via stdio transport and discovers available tools dynamically.

For product thinking, this matters because it changes the build-vs-integrate calculus entirely. Instead of building every capability into your agent, you compose them from MCP servers. Your agent becomes an orchestrator over an ecosystem of tools. The PM's job shifts from speccing individual features to **designing the orchestration layer**: which tools should the agent have access to, in what contexts, with what guardrails.

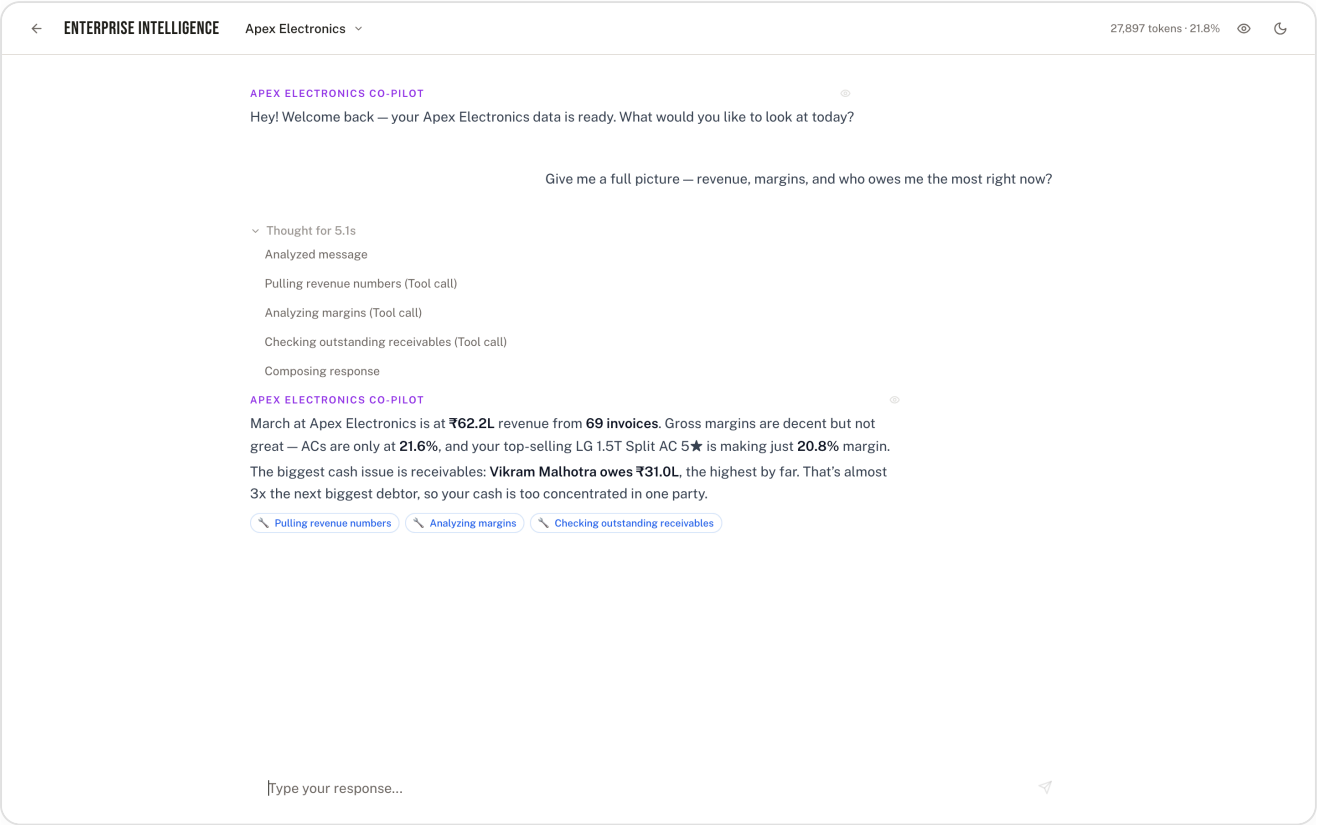
What's the one thing that separates you from other PMs working on AI?

I don't delegate the understanding. Most PMs learn AI concepts at the abstraction layer: they know what RAG is, they know what function-calling is, they can have the conversation. I **build the systems**. I've written the tool schemas, designed the embedding pipelines, debugged the token overflow issues, tuned the similarity thresholds, and shipped the streaming UX. When I make a product decision about an AI feature, it's grounded in having built it myself, not from a second-hand understanding of an engineer's explanation.

That doesn't mean I think PMs should be engineers. It means the best AI PMs right now are the ones who've **closed the gap** between what they can imagine and what they can validate. I can go from an idea to a working prototype in Cursor in a day. That speed isn't about coding, it's about having the judgment to know what to build, how the pieces fit, and where the real product risk is. And that only comes from building.

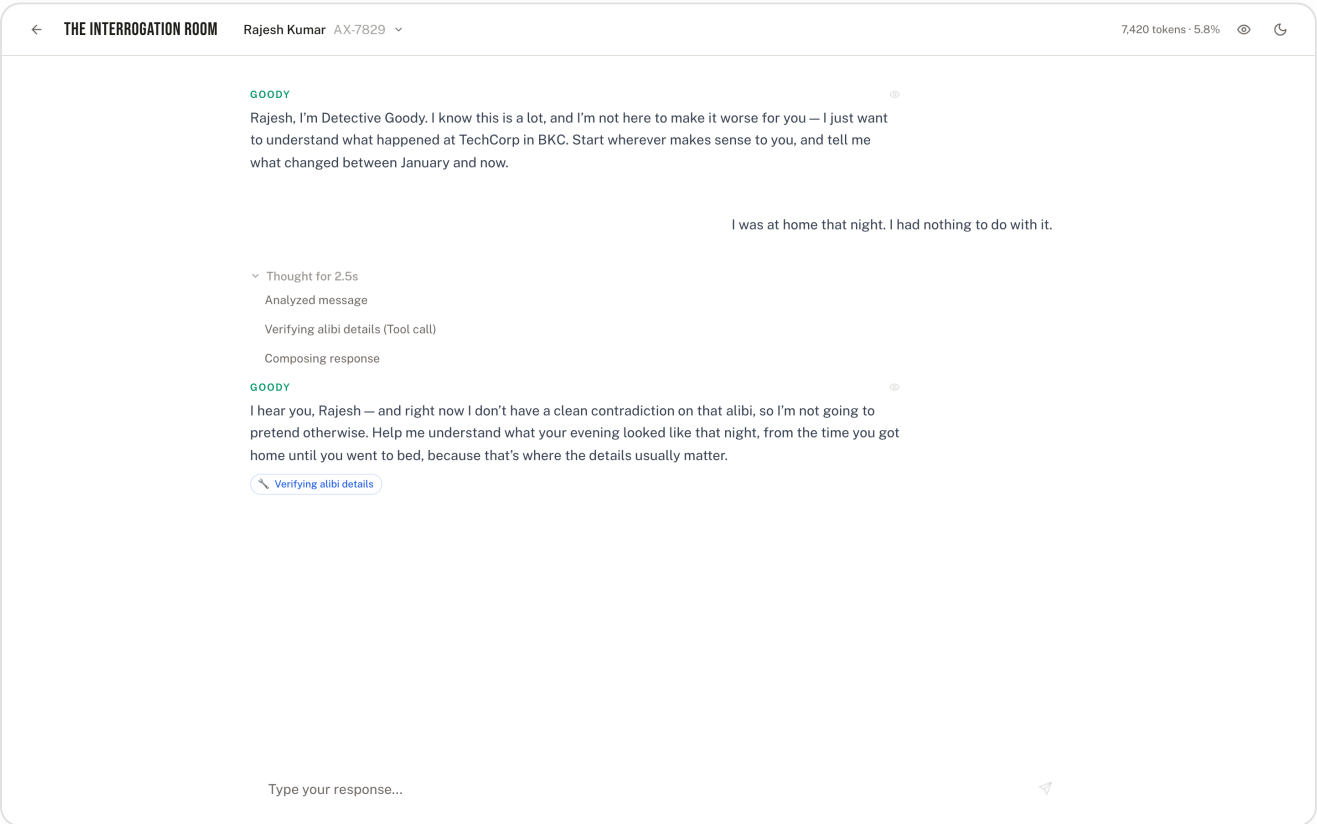
ENTERPRISE INTELLIGENCE

An AI business advisor that cross-references live transaction data with RAG-retrieved policies. The agent receives a broad question, reasons through it (5.1s), orchestrates **three parallel tool calls** (revenue, margins, receivables) and synthesizes a single opinionated response with ₹ figures and a concrete action.



THE INTERROGATION ROOM

A multi-agent Good Cop / Bad Cop system. Detective Goody opens with empathy, the suspect claims innocence, and the agent's expanded thought process shows the full pipeline: **analyze message → verify alibi (Tool call) → compose response**. The "Verifying alibi details" badge confirms the tool was invoked.



LLM OBSERVABILITY: TOOL TRACES

Every tool call is fully inspectable. Here the agent made **two successive calls**: **get_revenue_summary** (105ms) and **get_margin_analysis** (187ms), with input arguments and full JSON return values visible. The LLM Call panel shows 10,303 prompt tokens and 91 completion tokens.

ENTERPRISE INTELLIGENCE | Apex Electronics | 27,897 tokens · 21.8%

APEX ELECTRONICS CO-PILOT

Hey! Welcome back — your Apex Electronics data is ready. What would you like to look at today?

Give me a full picture — revenue, margins, and who owes me the most right now?

Thought for 5.1s

- Analyzed message
- Pulling revenue numbers (Tool call)
- Analyzing margins (Tool call)
- Checking outstanding receivables (Tool call)
- Composing response

APEX ELECTRONICS CO-PILOT

March at Apex Electronics is at **₹62.2L** revenue from **69 invoices**. Gross margins are decent but not great — ACs are only at **21.6%**, and your top-selling LG 1.5T Split AC 5★ is making just **20.8%** margin. The biggest cash issue is receivables: **Vikram Malhotra owes ₹31.0L**, the highest by far. That's almost 3x the next biggest debtor, so your cash is too concentrated in one party.

Pulling revenue numbers | Analyzing margins | Checking outstanding receivables

Type your response...

LLM Control Room

Turn: 1

Agent | Prompt | Tools | MCP | RAG | Pricing

LLM Call #1 ANALYZED MESSAGE 2198ms

PROMPT	COMPLETION	TOTAL
10,303	103	10,406

View messages sent to LLM (3)

get_revenue_summary SUCCESS 110ms

Pulling revenue numbers

INPUT ARGS

```
{
  "start_date": "2026-03-01",
  "end_date": "2026-03-29",
  "granularity": "monthly"
}
```

RETURN VALUE

```
{
  "granularity": "monthly",
  "periods": {
    "2026-03": {
      "revenue": 6221970.449999999,
      "discount": 94544.42999999996,
      "count": 69
    }
  }
}
```

get_margin_analysis SUCCESS 218ms

Analyzing margins

INPUT ARGS

```
{
  "start_date": "2026-03-01",
  "end_date": "2026-03-29"
}
```

LLM OBSERVABILITY: TOKEN ECONOMICS

Per-turn breakdown of where context budget goes: base system prompt (23%), tool definitions (87% of prompt tokens), dynamic context, RAG injection, conversation history, and output. Total: 22,200 tokens at ~$0.003/turn. Context window: 17.3% of 128K.

ENTERPRISE INTELLIGENCE | Apex Electronics | 27,897 tokens · 21.8%

APEX ELECTRONICS CO-PILOT

Hey! Welcome back — your Apex Electronics data is ready. What would you like to look at today?

Give me a full picture — revenue, margins, and who owes me the most right now?

Thought for 5.1s

- Analyzed message
- Pulling revenue numbers (Tool call)
- Analyzing margins (Tool call)
- Checking outstanding receivables (Tool call)
- Composing response

APEX ELECTRONICS CO-PILOT

March at Apex Electronics is at **₹62.2L** revenue from **69 invoices**. Gross margins are decent but not great — ACs are only at **21.6%**, and your top-selling LG 1.5T Split AC 5★ is making just **20.8%** margin. The biggest cash issue is receivables: **Vikram Malhotra owes ₹31.0L**, the highest by far. That's almost 3x the next biggest debtor, so your cash is too concentrated in one party.

Pulling revenue numbers | Analyzing margins | Checking outstanding receivables

Type your response...

LLM Control Room

Turn: 1

Agent | Prompt | Tools | MCP | RAG | Pricing

AUGMENTED SYSTEM PROMPT

Base system prompt	5,180 (23%)
Dynamic context	114 (1%)
RAG context (injected)	0 (0%)
= Augmented system prompt total	5,294
Tool definitions	19,298 (87%)
Conversation history (2 msgs)	0 (0%)
Completion (output)	232 (1%)

PROMPT TOKENS (API)	COMPLETION TOKENS
21,950	232
TOTAL TOKENS	**COST (THIS TURN)**
22,182	~$0.00333

Augmented System Prompt = Base Prompt + Dynamic Context + RAG Context. Conversation history is all messages so far. Output is the completion tokens for this turn.

Context window usage 17.3% of 128K